

Assignment 3: Controllers & Safety Filters

Due Apr. 20 at 23:59

This assignment is to be done individually.

Important Note: The university policy on academic dishonesty (cheating) will be taken very seriously in this course. You may not provide or use any solution, in whole or in part, to or by another student.

You are encouraged to discuss the concepts involved in the questions with other students. If you are in doubt as to what constitutes acceptable discussion, please ask! Further, please take advantage of office hours offered by the instructor and the TA if you are having difficulties with this assignment.

DO NOT:

- Give/receive code or proofs to/from other students.
- Copy solution directly from Google or ChatGPT.

DO:

- Meet with other students to discuss assignment (it is best not to take any notes during such meetings, and to re-work assignment on your own).
 - Use online resources (e.g. Wikipedia) to understand the concepts needed to solve the assignment.
 - Indicate in a cover note how AI tools were used, if at all, to help with the assignment.
-

Submitting Your Assignment

The assignment must be submitted on Canvas. Your submission must contain a zip file with the following files. These files must be in the root of the zip file.

- `lqr_algorithm.py`: LQR backend
 - `cbf_algorithm.py`: CBF backend
 - `hj_algorithm.py`: HJ reachability backend
 - `controller_node.py`: ROS2 front end
 - `{lqr, cbf, hj}.mp4`: videos of real-world experiments
-

1 Linear Quadratic Regulator Trajectory Tracking

In this question, you will implement a finite-horizon linear quadratic regulator (LQR) trajectory-tracking controller for a Dubins car moving in the plane.

The Dubins car dynamics are

$$\dot{p}_x(t) = v(t) \cos(\theta(t)), \quad (1a)$$

$$\dot{p}_y(t) = v(t) \sin(\theta(t)), \quad (1b)$$

$$\dot{\theta}(t) = \omega(t). \quad (1c)$$

The state is given by $z(t) = (p_x(t), p_y(t), \theta(t))$, where (p_x, p_y) is position and θ is heading. The control is given by $u(t) = (v(t), \omega(t))$, where v is the forward speed and ω is the turn rate.

The provided code includes a nominal reference trajectory for the robot. At each discrete time step t , the robot should track a state action reference sequence consisting of a nominal state and control functions

$$(z^{\text{ref}}(t), u^{\text{ref}}(t))$$

We will be using the commonly-adopted approach of discretizing the dynamics in time, and using the discrete-time formulation of LQR, specifically the formulation described in Prof. Pieter Abbeel's notes on page 22. This is a great set of notes and it would be helpful to go through most of it. We provide a very brief summary below.

Your task is to design an LQR controller that tracks a discretized reference trajectory by correcting deviations from the nominal motion by completing the LQR backend in `lqr_algorithm.py` and the ROS2 frontend in `controller_node.py`, which will be also used for the HJ Reachability and CBF parts of this assignment. Please see `readmd.md` for more detailed instructions.

What you need to implement

Complete the marked by `STUDENT CODE START` and `STUDENT CODE END` blocks in the provided code in `lqr_algorithm.py` and `controller_node.py`.

Notation

- Use the state ordering `[px, py, theta]`.
- Use the control ordering `[v, omega]`.
- The nominal state trajectory is denoted by `z_ref`.
- The nominal control trajectory is denoted by `u_ref`.
- A state and control trajectory is collected together inside the `nav_helps/msg/StateActionTrajectory` message. You may use the Python wrapper class, also named `StateActionTrajectory`.

- The time step is denoted by `dt`.
- The controller horizon is denoted by `horizon`.
- The state-cost, control-cost, and terminal-cost matrices are denoted by Q , R , and L .
- In the ROS2 node, the latest measured state is stored as `[x, y, yaw]`.

Hints

- Make sure to use the state ordering `[px, py, theta]` and control ordering `[v, omega]`.
- Linearize the dynamics around the nominal pair $(z_t^{\text{ref}}, u_t^{\text{ref}})$ at each time step.
- Use forward Euler discretization, not an exact matrix exponential.
- Initialize the backward recursion using the terminal cost matrix L .
- The tracking controller should use the nominal control together with a feedback correction.
- Be careful to wrap the heading error before applying feedback.
- In receding-horizon control, only the first control action from the computed sequence should be returned.
- The planner node should never crash if the backend fails; it should publish a safe zero command instead.
- A correct implementation should track the reference smoothly while keeping the control inputs finite and well behaved.

1.1 Summary of Discrete-Time LQR

1.1.1 Tracking objective

Define the tracking errors

$$\delta_{z_t} = z_t - z_t^{\text{ref}}, \quad \delta_{u_t} = u_t - u_t^{\text{ref}}.$$

In this assignment, you will use the finite-horizon quadratic objective

$$J = \sum_{t=0}^{N-1} (\delta_{z_t}^\top Q \delta_{z_t} + \delta_{u_t}^\top R \delta_{u_t}) + \delta_{z_N}^\top L \delta_{z_N},$$

where Q penalizes state tracking error, R penalizes deviation from the nominal control, and L penalizes terminal tracking error.

The cost matrices are taken to be diagonal:

$$Q = \text{diag}(q_x, q_y, q_\theta), \quad R = \text{diag}(r_v, r_\omega), \quad L = \text{diag}(\ell_x, \ell_y, \ell_\theta).$$

1.1.2 Linearization and discretization

Because the Dubins car dynamics are nonlinear, you must linearize them around the nominal trajectory. At each time step t , you should form a local continuous-time linear model of the error dynamics around $(z_t^{\text{ref}}, u_t^{\text{ref}})$, and then convert it to a discrete-time approximation using forward Euler with time step Δt .

Your linearized discrete-time error dynamics should have the form

$$\delta_{z_{t+1}} \approx A_t \delta_{z_t} + B_t \delta_{u_t}.$$

1.1.3 Finite-horizon LQR recursion

Using the matrices A_t and B_t , your implementation should compute a sequence of feedback gains K_t by running the finite-horizon Riccati recursion backward from the terminal cost.

The resulting control law should have the form

$$u_t = u_t^{\text{ref}} - K_t \delta_{z_t}.$$

In other words, your controller should remain close to the nominal control when tracking is already good, and should apply a feedback correction when the robot deviates from the nominal trajectory.

1.1.4 Tracking error representation

In this assignment, special care is needed when computing the heading error. Since angles are periodic, directly subtracting two headings can produce an incorrect large error near the branch cut at $\pm\pi$.

Therefore, when constructing the tracking error, you should represent the heading error using a wrapped angle difference so that the angular error always lies in $(-\pi, \pi]$.

1.1.5 Receding-horizon execution

At each control step, the controller should:

- (i) select the current finite reference window,
- (ii) compute the linearized model and LQR gains over that window,
- (iii) compute the corresponding tracking controls,
- (iv) apply only the first control input in closed loop.

2 Control Barrier Function Safety Filter

In this question, you will implement a control barrier function (CBF) safety filter for a Dubins car moving through a 2D corridor with a circular obstacle.

The Dubins car dynamics are

$$\dot{p}_x(t) = v(t) \cos(\theta(t)), \quad (2a)$$

$$\dot{p}_y(t) = v(t) \sin(\theta(t)), \quad (2b)$$

$$\dot{\theta}(t) = \omega(t). \quad (2c)$$

The state is given by $x(t) = (p_x(t), p_y(t), \theta(t))$, where (p_x, p_y) is position and θ is heading. The control is given by $u(t) = (v(t), \omega(t))$, where v is the forward speed and ω is the turn rate.

In the defaults for python-only backend `cbf_algorithm`, the robot must remain inside the corridor

$$y_{\text{low}}(p_x(t)) \leq p_y(t) \leq y_{\text{high}}(p_x(t)), \quad (3a)$$

$$x_{\text{min}} \leq p_x(t) \leq x_{\text{max}}, \quad (3b)$$

using bounded controls

$$0 \leq v(t) \leq 1.0, \quad (4a)$$

$$-1.2 \leq \omega(t) \leq 1.2. \quad (4b)$$

The corridor is defined by piecewise-linear interpolation through the following knot points:

$$x_{\text{knots}} = [0.0, 2.0, 4.0, 6.0, 8.0],$$

$$y_{\text{low}} = [-1.0, -1.4, -1.6, -1.6, -1.4],$$

$$y_{\text{high}} = [1.0, 1.4, 1.6, 1.6, 1.4].$$

The initial state is

$$x_0 = [0.5 \quad 0.0 \quad 0.0]^\top,$$

and the goal state is

$$g = [7.5 \quad 0.0 \quad 0.0]^\top.$$

There is one circular obstacle inside the corridor, with center

$$c = \begin{bmatrix} 4.3 \\ 0.15 \end{bmatrix}, \quad r_{\text{obs}} = 0.50,$$

and a desired safety margin

$$d_{\text{safe}} = 0.20.$$

The provided code includes a nominal nonlinear MPC controller that drives the robot toward the goal while respecting the corridor and input constraints. However, the nominal controller does *not* account for the obstacle. In the starter code, the nominal control

$$u_{\text{nom}} = \begin{bmatrix} v_{\text{nom}} \\ \omega_{\text{nom}} \end{bmatrix}$$

is already provided to the safety filter at each time step.

Your task is to implement a CBF-based safety filter that modifies this nominal control only when necessary for obstacle avoidance.

Barrier design In class, we covered computation of CBFs using optimization-based methods (e.g. sum-of-squares optimization); other methods such as solving a modified HJ PDE also exist. In this assignment, however, we will simplify, and handcraft a CBF and validate it empirically.

You will design your barrier using a *look-ahead point* rather than the robot center. Because of the nonholonomic kinematics of the Dubins car, a distance-based barrier defined only from the robot center (p_x, p_y) does not allow the steering input ω to appear properly in the first-order CBF constraint. As a result, the safety filter would not be able to use steering effectively for obstacle avoidance.

Therefore, define a look-ahead point

$$p_L(x) = \begin{bmatrix} p_x + L \cos \theta \\ p_y + L \sin \theta \end{bmatrix},$$

where you should use

$$L = 0.8.$$

You may also use the provided buffer

$$\varepsilon = 0.05$$

when defining an effective safety radius or margin. You should decide how to define the barrier $h(x)$ so that it represents safe separation from the circular obstacle while accounting for the geometry induced by the look-ahead point.

CBF condition. Your barrier should lead to a safety condition of the form

$$\dot{h}(x, u) + \gamma h(x) \geq 0, \quad \gamma = 2.0.$$

In the provided implementation, you should compute quantities $h(x)$, $a(x)$, and $b(x)$ such that your CBF condition can be written as

$$a(x)v + b(x)\omega + \gamma h(x) \geq 0.$$

In other words, your `cbf_terms(...)` function should return values h , a , and b so that the CBF constraint above is meaningful for your chosen barrier design.

Safety filter QP. At each time step, the safety filter should compute a filtered control

$$u^* = \begin{bmatrix} v^* \\ \omega^* \end{bmatrix}$$

by solving the quadratic program

$$\begin{aligned} u^* = \arg \min_{u \in \mathbb{R}^2} \quad & \frac{1}{2} (w_v(v - v_{\text{nom}})^2 + w_\omega(\omega - \omega_{\text{nom}})^2) \\ \text{s.t.} \quad & a(x)v + b(x)\omega + \gamma h(x) \geq 0, \\ & 0 \leq v \leq 1.0, \\ & -1.2 \leq \omega \leq 1.2, \end{aligned}$$

where

$$w_v = 1.0, \quad w_\omega = 0.05.$$

This choice of weights makes changing the turn rate cheaper than changing the speed, so that the safety filter prefers steering around the obstacle rather than simply stopping in front of it.

Your task is to finish the CBF-related parts of the implementation in `cbf_algorithm.py`.

Notation

- Many parameters, including control bounds, weights, and CBF parameters, are found in the dictionary `params`. For example, γ corresponds to `params["gamma_cbf"]`.
- The corridor parameters are specified in `corridor_params`.
- The obstacle parameters are specified in `obstacle`.
- The goal state is specified by `goal`.
- The look-ahead distance corresponds to `params["lookahead_distance"]`.
- The additional buffer corresponds to `params["center_margin_buffer"]`.

What you need to implement

Complete the marked by `STUDENT CODE START` and `STUDENT CODE END` blocks in the provided code in `cbf_algorithm.py`.

Run the provided python-only simulation to visualize the resulting trajectory and barrier-related plots.

Hints

- Make sure to use the state ordering `[px, py, theta]` and control ordering `[v, omega]`.
- The nominal control input is already provided to the safety filter in the starter code.
- You must define your barrier using the look-ahead point, not the robot center.
- Your barrier must be designed so that steering appears in the affine CBF constraint.
- Your `cbf_terms(...)` function must return three finite scalar values: h , a , and b .
- Your safety filter should always return a valid bounded control input.
- The safety filter should stay close to the nominal control when the nominal control is already safe.
- A correct implementation should keep the robot away from the obstacle while still allowing it to make progress toward the goal.

3 Reachability Analysis as Safety Filter

Control Barrier Functions require us to engineer a good $h(x)$ that guarantees safety invariance, as can be seen in the previous section. The region of $h(x) \geq 0$ encodes the safety set of the system. In this question, we will obtain such region using reachability analysis instead. Reachability analysis tool can help us avoid constructing such function manually. For now, we will use the same corridor settings and same dynamics as previous questions (initial state, goal, obstacle position, etc) except now we change the speed's bound to $0.5 \leq v(t) \leq 1$. This question comprises a mix of hand-written answer and coding tasks.

You will compute the following value function as seen in the lecture slide:

$$V(x(t), t) = \max_{u(\cdot)} \min_{s \in [t, 0]} l(x(s)) \quad (5)$$

1. Same as before, we want our system to avoid the circular obstacle with center

$$c = \begin{bmatrix} 4.3 \\ 0.15 \end{bmatrix}, \quad r_{\text{obs}} = 0.50,$$

and a desired safety margin

$$d_{\text{safe}} = 0.20.$$

What would the function $l(x)$ be that encodes the current state x collides with the obstacle when $l(x) \leq 0$ and safe otherwise. Write it down explicitly. *Hint: Consider distance to the center of the obstacle.* Quickly explain how the setup of this function aligns with the objective of the control function appearing in Eq. (5).

Additionally, if $l(x(t)) > 0$, is it necessarily true that $V(x(t), t) > 0$? What does this imply in terms of safety?

2. The next task you need to do is computing the value function V . It has been shown that V is the solution to the following Hamilton-Jacobi partial differential equation (PDE):

$$\min \left\{ \frac{\partial V}{\partial x}(x, t) + \max_{u \in \mathcal{U}} \frac{\partial V}{\partial x}(x, t)^\top f(x, u), l(x) - V(x, t) \right\} = 0 \quad (6)$$

$$V(x, 0) = l(x)$$

If we solve this pde, we basically solve for V . Don't worry if this looks intimidating! We have already provided a software library to solve this particular class of pde here: https://github.com/SFU-MARS/optimized_dp. Please follow the instructions to install this toolbox inside a **conda** environment in your docker container. Like most PDEs, HJ PDE is also solved numerically on a grid. What uniquely defines this problem are the initial value function $l(x)$ and the system dynamics $f(x, u)$.

Before computing the value function, write out explicitly the analytic expression of optimal speed and heading angle. That is, find an analytic expression for u^* , where

$$u^*(t, x) = \arg \max_{u \in \mathcal{U}} \frac{\partial V}{\partial x}(x, t)^\top f(x, u) \quad (7)$$

To obtain V for this problem, you need to specify the problems correctly when implementing your change in the provided file `py/compute_hj_value_function.py`. You can refer to the following example to get started https://github.com/SFU-MARS/optimized_dp/blob/master/examples/pursuit_evasion_game.py

In your implementation, please use the following recommended number of grid points in each dimension. This ensures sufficient accuracy, and consistency in assignment submissions.

- p_x : 100 grid points
- p_y : 100 grid points
- θ : 72 grid points (periodic)

Requirements:

- Choose a grid bound large enough to contain the zero sub-level set $\mathcal{S}^- = \{x : V(-T, x) \leq 0\}$. We also denote the surface boundary as $\partial\mathcal{S} = \{x : V(-T, x) = 0\}$
- Usually to guarantee safety, we would solve V backward in time until convergence $V_{t \rightarrow -\infty}(x(t), t)$. Choose the time horizon to be $t \in [-T, 0]$ for a T large enough for $V(t, x)$ to converge.
- We have already provided a system dynamics object `DubinsCar2`, you can use it in your implementation and ignore the disturbance by setting them to zeros
- Run the solver and generate an image of $\partial\mathcal{S}$ using the following code :

```
po2 = PlotOptions(do_plot=False, plot_type="set",
                  plotDims=[0, 1, 2],
                  slicesCut=[], save_fig=True,
                  filename="test_full.png")
visualize_plots(result, g, po2)
```

- If you're not running inside Docker container, you can also try set `do_plot=True` and interact with the plot on your browser
- Plot a 2D slice of $\partial\mathcal{S}$ at $\theta = [-\pi, -\pi/2, 0, \pi/2]$ by using `PlotOptions` and setting the parameters `plotDims` and `slicesCut`
- Save the grid information, value function and the spatial derivatives for all state components into `.npz` or `.npy` file format so that we can re-use them in later sections.

Bonus: What is the time and space complexity of solving this PDE?

- Now that we have computed V , we can now use it as a filter that safeguard our nominal controller just like the QP program. We can ensure the system is safe by always keep it inside the zero super-level set $\mathcal{S}^+ = \{x : V(-T, x) > 0\}$. This set is forward invariant. In short, this means that while state x is still inside $\mathcal{S}^+$, the optimal control obtained from (7) will keep it inside this set.

Most of the time, we don't need to apply this control but rather the nominal control. And

only when x is near the boundary of $\mathcal{S}^+$, we would intervene with the optimal control to keep it inside $\mathcal{S}^+$. This logic is often called *Least Restrictive Switching Filter*:

$$u = \begin{cases} u^* \text{ from (7)} & \text{if } V(-T, x) \leq \epsilon, \\ u_{\text{nom}} & \text{otherwise} \end{cases} \quad (8)$$

where ϵ is a small positive number, which is chosen to be $\epsilon = 0.1$ for this part. Provide your implementation for this filter in the provided file `hj_algorithm.py` in the `controller` package. This will require you to load the save `.npy` and `.npz` file computed from previous question.

What you need to implement

Complete the marked by `STUDENT CODE START` and `STUDENT CODE END` blocks in the provided code in `hj_algorithm.py` and `compute_hj_value_function.py`.

You can run the file `hj_algorithm.py` separately to visualize the resulting trajectory to see the safety filter in effect.

Hints

- Make sure to use the state ordering `[px, py, theta]` and control ordering `[v, omega]`.
- The nominal control input is already provided to the safety filter in the starter code.
- To complete `hj_algorithm.py`, you can refer to https://github.com/SFU-MARS/optimized_dp/blob/master/examples/pursuit_evasion_game.py for useful example.
- You will find the python member functions of class `Grid` and `DubinsCar2` useful for implementing the safety filter switch logic
- It's important to notice that `theta` is periodic, make sure that you can specify this when initializing the grid in `compute_hj_value_function.py`. This will ensure the solver can correctly handle the periodicity of the state space.
- To plot the 2D slice of $\partial\mathcal{S}$, you can set `plotDims=[0, 1]` and input in index for the third dimension in `slicesCut`.

4 Hardware Experiments on the TurtleBot3

Now that everything works in simulation, it's time to test in the real world by following these steps:

1. Set up an environment that can be well-described by the corridor constraints.
2. Map this environment, and start the robot in the desired initial position.
3. If using `CBFController` or `HJController`, place an additional obstacle in the environment.
4. Carefully specify the parameters of the planner and controller to match the real world environment.
5. Set up the robot and networking.
6. Launch `robot.launch.py` on the TurtleBot3, and `controller.launch.py` in the container on the external computer.

A lot more details can be found in `robot_readme.md`.

Record your real-world experiment by taking three single uncut/unedited video showing

- the terminal from which `controller.launch.py` is launched
- the rviz window that pops up,
- the robot navigating in its environment.

Name these videos `{lqr, cbf, hj}.mp4` and include them in your submission package.